

HW 6: Problem 1.1 Analysis

Problem 1.1: Strategic Network Formation

Part 1: Undirected Graph with Indicator Payoffs (Linear Model)

In the undirected model with indicator payoffs (referred to as the “Linear Model”), both participants of an edge pay the cost c because an edge $\{i, j\}$ increases both $|N_i|$ and $|N_j|$. The benefit of reaching another node does not decay with distance; it is simply 1 if a path exists. As seen in the code listing in Appendix A, we simulate these conditions starting from four distinct initial graph topologies: Empty, Complete, Star, and Cycle graphs.

- **Regime $c < 1$:** We simulated $c = 0.5$. The equilibrium reached from all initial states is a tree (8 edges for 9 nodes), which perfectly connects all nodes without any redundant edges. There does not appear to be a *unique* equilibrium graph topology (any spanning tree is an equilibrium because removing an edge breaks the component and losing ≥ 1 reachability is worse than saving $c = 0.5$, while adding an edge yields 0 additional reachability for cost c). The resulting trees are highly efficient since they maximize reachability at the minimum possible total edge cost. The total social welfare is mathematically $\Pi(\text{Tree}) = \sum_i (n-1) - 2c \cdot |E| = n(n-1) - 2c(n-1)$. Since any spanning tree (including a star graph) has exactly $|E| = n-1$ edges and $\sum_i (n-1)$ reachability, they all yield the exact same mathematically optimal social welfare of $72 - 8 = 64$. A star is functionally equivalent to a line or any other tree in this specific non-decaying indicator payoff model.
- **Regime $1 < c < n-1$:** We simulated $c = 4.0$. From all starting configurations (Empty, Complete, Star, Cycle), the simulation converges to the *empty graph* (0 edges). This is a unique equilibrium because any terminal node (a node with degree 1) pays $c > 1$ to maintain its single edge, but only gains a direct benefit of 1 from connecting to its neighbor. Even if that neighbor provides transitive reachability to the rest of the network, the terminal node’s net marginal benefit for that edge is at most $1 + (n-2) = n-1$. However, the hub node connecting it must also pay $c > 1$ for that specific edge. Since the hub node is already connected to the rest of the network, that specific edge to the terminal node only provides a marginal reachability benefit of exactly 1. Since $c > 1$, the hub node will strictly prefer to sever the tie ($\Delta\text{Payoff} = c - 1 > 0$), causing the network to systematically unravel from the leaves inward until 0 edges remain. It is severely inefficient. For instance, a centralized star topology would have a socially optimal net positive welfare of $\Pi(\text{Star}) = n(n-1) - 2c(n-1) = 72 - 64 = 8$, but the empty graph leaves us at $\Pi(\text{Empty}) = 0$.
- **Regime $c > n-1$:** We simulated $c = 10.0$. The unique equilibrium is the *empty graph*. Because $c > 8$, the maximum possible benefit of playing an edge (reaching all 8 other nodes) is still strictly less than the cost of a single edge ($c = 10$). The empty network is efficient in this regime because the maximum conceivable system-wide benefit of an edge (a cycle yielding $2(n-1)$ to the participants and perhaps some transitive benefit to others) is strictly outweighed by the direct costs. Formally, for any network G , $\Pi(G) \leq n(n-1) - 2c \cdot |E| < 0$ since $c > n-1$ means $2c > 2(n-1) \geq n$ (for $n > 2$). Thus $\Pi(\text{Empty}) = 0$ is optimal.

As shown in Figure 1, the equilibrium topography efficiently links every node.

Part 2: Directed Graph with Indicator Payoffs (Linear Model)

In the directed setting, nodes only pay for their outgoing edges, resolving the bilateral cost-sharing resistance seen in the undirected case. However, unlike undirected edges where a single link $\{A, B\}$ provides mutual two-way access, a directed edge (A, B) only provides A access to B , not vice-versa. This implies that a tree topology ($n-1$ edges), which perfectly connects an undirected graph, cannot provide full reachability in a directed graph because there is no path back to the roots. The minimal structure to achieve full global reachability (strong connectivity) is a directed cycle (n edges).

- **Regime $c < 1$:** We simulated $c = 0.5$. Because $c < 1$, any unreached node provides 1 benefit for only c cost, prompting nodes to form links until the graph is strongly connected. A perfect directed cycle (9 edges for 9 nodes) is an efficient equilibrium with a maximal social welfare of $\Pi(\text{Cycle}) = \sum_i (n-1-c \cdot 1) = n(n-1-c) = 9(8-0.5) = 67.5$. The cycle is the most efficient topology because it provides every node full indicator reachability (+8) using the absolute mathematical minimum number of directed edges ($n = 9$). This shifts the optimal structure from an $n-1$ edge tree in Part 1 to an n edge cycle in Part 2, a direct consequence of the one-way nature of directed traversal. While

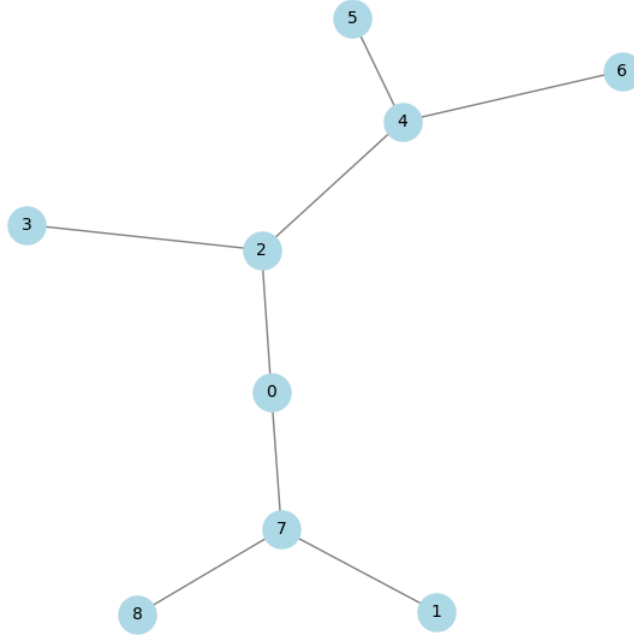


Figure 1: Equilibrium for Undirected Linear ($c < 1$). The network efficiently connects all nodes via a tree without any redundant edges.

the simulation sometimes converged to local Nash Equilibria with slightly more non-redundant edges depending on initialization, the cyclical motif remains paramount for reachability.

- **Regime $1 < c < n - 1$:** We simulated $c = 4.0$. The equilibria are deeply dependent on the initialization. If started from an empty graph or an inward star, no node wants to pay $c = 4.0$ to reach just 1 node, so the *empty graph* remains a stable equilibrium (inefficient, $\Pi = 0$). Conversely, if initialized as a complete graph, outward star, or cycle, the network trims down precisely to a *directed cycle* (9 edges). A cycle is a Nash Equilibrium because each node pays $c = 4.0$ for 1 outgoing edge but achieves a reachability of $8 \times 1 = 8$, netting $+4.0$. The cycle is an efficient topology here, achieving the maximum welfare $\Pi = n(n - 1 - c) = 36$.
- **Regime $c > n - 1$:** We simulated $c = 10.0$. As in the undirected case, the cost of a single edge exceeds the absolute maximum reachability benefit of $n - 1 = 8$. Thus, all nodes rapidly sever all outgoing edges, leaving the *empty graph* as the unique and efficient equilibrium ($\Pi = 0$).

Figure 2 visualizes this cyclical stability.

Part 3: Undirected Graph with Distance Payoffs (Connections Model)

In the connections model, the benefit explicitly decays with distance ($\delta = 0.5$).

- **Regime $c < \delta - \delta^2$:** We tested $c = 0.1$. As seen in Figure 3, the simulation instantly converges from any initialization to the *complete graph* (36 edges). Since $c < 0.25$, any node prefers adding a direct edge to another node rather than having an indirect path of distance 2 or more, as the gain ($\delta - \delta^2 = 0.25$) exceeds the cost c . Thus, it is the unique equilibrium and is efficient. The welfare mathematically is $\Pi(\text{Complete}) = n(n - 1)(\delta - c) = 9(8)(0.5 - 0.1) = 28.8$.
- **Regime $\delta - \delta^2 < c < \delta + (n/2 - 1)\delta^2$:** We tested $c = 0.8$. Starting from an empty graph (where no nodes share any edges), the simulation trivially terminates because adding the very first edge costs 0.8 but only provides a direct marginal benefit of $\delta = 0.5$. This empty equilibrium is rigorously inefficient ($\Pi = 0$).

However, if we initialize the simulation from dense, highly-connected topologies (such as the Complete graph, Star graph, or Cycle graph), the simulation enters continuous cyclic behaviors (plotted in Figure 4). This instability occurs because nodes now disagree on the network's value depending on their algorithmic structural position. Peripheral nodes strictly benefit from finding cheap, short paths through central "hubs", so they aggressively desire to form or maintain links. But the central hub itself pays immense, compounding link costs (-0.8 per edge) without receiving equivalent distance payoffs (since it already had short paths to most nodes). As a result, the hub inevitably severs its ties to save

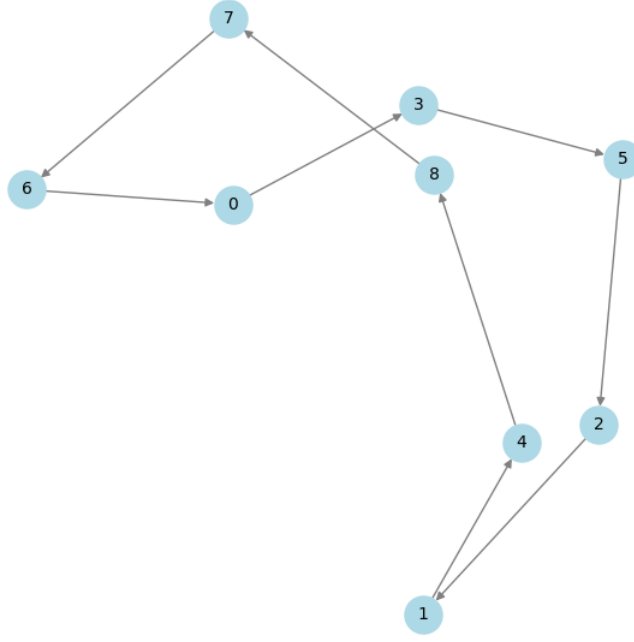


Figure 2: Equilibrium for Directed Linear ($1 < c < n - 1$) starting from a dense graph. The result trims precisely to an efficient directed cycle.

cost. This fractures the network, raising distances for the peripheral nodes, who then frantically try to build new edges to reconnect—causing the cycle to endlessly repeat without finding a stable Nash Equilibrium. This regime highlights a stark conflict between Nash equilibrium stability and social welfare: the socially optimum topology (the Star graph) has a strongly positive efficiency ($\Pi = 8(2(0.5) - 2(0.8)) + 56(0.5^2) = -4.8 + 14 = 9.2$), but no individual hub node will stably shoulder the personal cost to maintain it. Consequently, the network is either trapped at the 0 efficiency empty state or forced to endlessly cycle through suboptimal, transient inefficiencies.

- **Regime $c > \delta + (n/2 - 1)\delta^2$:** We tested $c = 2.0$. It firmly resolves back to the *empty graph* from almost all common starting points. Edge costs systematically overpower proximity rewards on all structural vectors ($2.0 > 0.5 + (3.5) \cdot 0.25 = 1.375$), so nodes aggressively drop edges. The empty graph is uniquely efficient $\Pi = 0$ and acts as the predominant strong stable equilibrium here.

Figure 3 shows the uniform clustering of edges when direct distances are heavily rewarded, while Figure 4 demonstrates the cyclic instability in the intermediate cost regime.

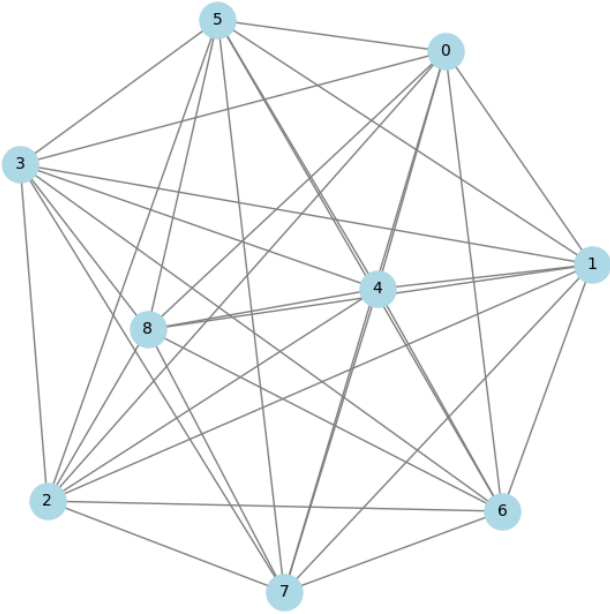


Figure 3: Equilibrium for Undirected Connections ($c < \delta - \delta^2$). Penalizing indirect paths quickly settles it into a complete graph.

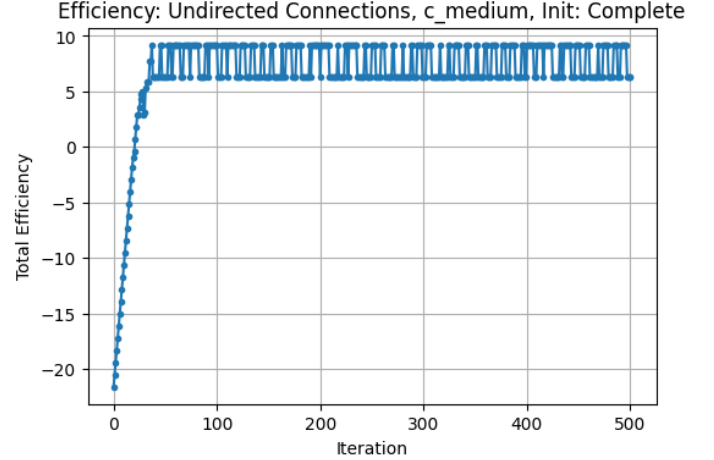


Figure 4: Efficiency convergence for Undirected Connections ($c = 0.8$) initialized from Complete graph. The system cycles continuously without a stable equilibrium.

Problem 1.2: Escaping the Cursed Maze

1. Why BFS/DFS Fails

Standard graph search algorithms like Depth-First Search (DFS) or Breadth-First Search (BFS) rely on a deterministic transition framework: if a node attempts to move North, it implicitly trusts that the resulting state will strictly be the room to the North. However, the cursed maze introduces a $p = 0.20$ probability of “slipping” on a conveyor belt, forcing the agent into a random adjacent room. This stochastic transition matrix destroys the absolute viability of static paths. Even if BFS finds the absolute shortest sequence of cardinal moves to an exit, it cannot guarantee that sequence will actually be executed. Crucially, algorithms like BFS fail to internalize risk: a shortest path that skirts tightly along the edge of multiple snake rooms is astronomically dangerous under a 20% slip rate, but BFS cannot distinguish this probabilistic danger from a safe open corridor as long as the step count is identical.

2. TD(0) Learning and Escaping

To successfully navigate the maze, we modeled the problem as Reinforcement Learning over a Markov Decision Process, specifically utilizing a Temporal Difference TD(0) value update rule. We learned the state value function $V(s)$ by exploring the maze for 998 episodes using an ϵ -greedy policy to balance exploiting known safe paths and exploring unknown areas.

Hyperparameters Configured:

- **Learning Rate** $\alpha = 0.5$: A moderate learning rate to quickly backpropagate terminal signals while smoothing out the noise from stochastic collisions.
- **Exploration** ϵ : Decayed exponentially from 1.0 down to 0.05 over the 998 episodes using a 0.995 multiplier per episode, ensuring broad early exploration that gradually settles into confident exploitation.
- **Rewards**: Snakes were initialized and permanently fixed at -100.0 , escapes fixed at $+100.0$. Standard, unexplored square rooms began at 0.0.

During our empirical test consisting of 1000 iterations with a purely greedy policy ($\epsilon = 0$), the agent successfully escaped the maze **1000 out of 1000** times, achieving a 100% survival rate. Because the agent directly learned $V(s)$ from stochastic experience, it implicitly learned to heavily discount the values of rooms directly physically adjacent to snakes—punishing the areas where slipping would be lethal. This phenomenon is directly supported by Figure 5, which shows distinct low-value suppression zones surrounded by the ‘S’ rooms. Thus, the greedy policy naturally prefers longer, safer corridors over dangerous shortcuts.

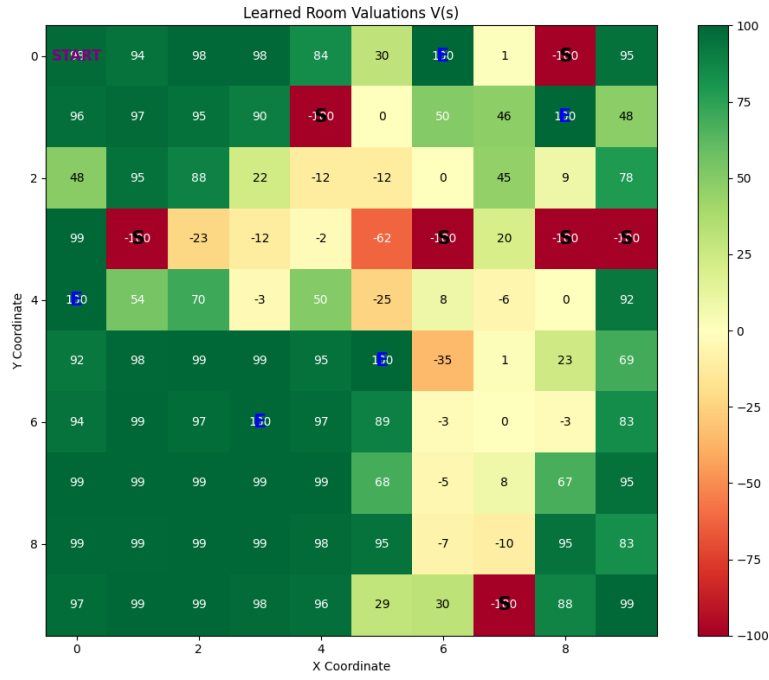


Figure 5: The converged $V(s)$ state-value grid after 998 episodes. Safest, most attractive rooms radiate outward from the escapes (E), while regions bordering snakes (S) are heavily suppressed.

The fully converged state-valuation grid $V(s)$ that drives the optimal policy is plotted in Figure 5. The complete Python implementation for learning and solving the maze (`solve_maze.py`) is provided in Appendix A.

Problem 2: Theory

2.1 Routing Games with Tolls

We are given a routing game with N players, each with traffic $r_i = 1$. The original link cost function is $c_e(x)$ and the original objective function is the social cost:

$$W(f) = \sum_{e \in E} f_e c_e(f_e)$$

where f_e is the total traffic (number of players) on edge e . We introduce a new cost function with a toll:

$$c_e^*(x) = c_e(x) + (x-1)(c_e(x) - c_e(x-1))$$

We must prove that the flow f^* that minimizes the original social cost $W(f)$ is a pure Nash Equilibrium for the new game with costs $c_e^*(x)$.

To prove this, we construct the exact potential function $\Phi^*(f)$ for the new game. For any routing game with integer flows and edge costs $c_e^*(x)$, the standard Rosenthal potential function given is:

$$\Phi^*(f) = \sum_{e \in E} \sum_{x=1}^{f_e} c_e^*(x)$$

Let us substitute the new cost function $c_e^*(x)$ into this potential function and simplify the inner sum for a single edge e :

$$\begin{aligned} \sum_{x=1}^{f_e} c_e^*(x) &= \sum_{x=1}^{f_e} [c_e(x) + (x-1)c_e(x) - (x-1)c_e(x-1)] \\ &= \sum_{x=1}^{f_e} [xc_e(x) - (x-1)c_e(x-1)] \end{aligned}$$

Notice that this inner sum is a perfect telescoping series! Let $g_e(x) = xc_e(x)$. The sum becomes:

$$\begin{aligned} \sum_{x=1}^{f_e} [g_e(x) - g_e(x-1)] &= (g_e(1) - g_e(0)) + (g_e(2) - g_e(1)) + \cdots + (g_e(f_e) - g_e(f_e-1)) \\ &= g_e(f_e) - g_e(0) \end{aligned}$$

Since $g_e(0) = 0 \cdot c_e(0) = 0$, we simply have:

$$\sum_{x=1}^{f_e} c_e^*(x) = f_e c_e(f_e)$$

Substituting this back into the full potential function $\Phi^*(f)$:

$$\Phi^*(f) = \sum_{e \in E} f_e c_e(f_e) = W(f)$$

We have shown that the exact potential function $\Phi^*(f)$ of the new game with tolls is mathematically identical to the social cost objective function $W(f)$ of the original game.

By the fundamental property of potential games, any pure Nash Equilibrium is a local minimum of the potential function. More importantly, the global minimum of the potential function is guaranteed to be a pure Nash Equilibrium. Since f^* is the optimal strategy that globally minimizes $W(f)$, and $W(f) = \Phi^*(f)$, f^* globally minimizes the potential function of the new game. Therefore, f^* must be a Nash Equilibrium for the new game.

A Code Listing

```
1 import networkx as nx
2 import numpy as np
3 import matplotlib.pyplot as plt
4
5 np.random.seed(42)
6
7 def linear_payoffs(G, c, graph_type='undirected'):
8     n = G.number_of_nodes()
9     payoffs = np.zeros(n)
10    for i in range(n):
11        if graph_type == 'undirected':
12            ni = G.degree(i)
13            reach = len(nx.node_connected_component(G, i)) - 1
14        else:
15            ni = G.out_degree(i)
16            reach = len(nx.descendants(G, i))
17
18        payoffs[i] = -c * ni + reach
19    return payoffs
20
21 def connections_payoffs(G, c, delta, graph_type='undirected'):
22     n = G.number_of_nodes()
23     payoffs = np.zeros(n)
24     path_lengths = dict(nx.shortest_path_length(G))
25     for i in range(n):
26         if graph_type == 'undirected':
27             ni = G.degree(i)
28         else:
29             ni = G.out_degree(i)
30
31         benefit = 0
32         for j in range(n):
33             if i != j:
34                 if j in path_lengths[i]:
35                     d = path_lengths[i][j]
36                     benefit += delta ** d
37
38         payoffs[i] = -c * ni + benefit
39    return payoffs
40
41 def get_possible_actions(G, u, graph_type='undirected'):
42     actions = [('none', None)]
43     n = G.number_of_nodes()
44
45     if graph_type == 'undirected':
46         neighbors = set(G.neighbors(u))
47     else:
48         neighbors = set(G.successors(u))
49
50     non_neighbors = set(range(n)) - neighbors - {u}
51
52     for w in non_neighbors:
53         actions.append(('add', w))
54
55     for v in neighbors:
56         actions.append(('remove', v))
57
58     for v in neighbors:
59         for w in non_neighbors:
60             actions.append(('swap', (v, w)))
61
62     return actions
63
64 def apply_action(G, u, action, graph_type='undirected'):
65     G_new = G.copy()
66     act_type, params = action
67
68     if act_type == 'none':
69         pass
70     elif act_type == 'add':
71         G_new.add_edge(u, params)
72     elif act_type == 'remove':
73         G_new.remove_edge(u, params)
```

```

74     elif act_type == 'swap':
75         v, w = params
76         G_new.remove_edge(u, v)
77         G_new.add_edge(u, w)
78
79     return G_new
80
81 def run_simulation(init_G, c, model='linear', delta=0.5, graph_type='undirected', max_iters=500):
82     G = init_G.copy()
83     n = G.number_of_nodes()
84
85     history = [tuple(sorted(G.edges()))]
86
87     if model == 'linear':
88         eff = sum(linear_payoffs(G, c, graph_type))
89     else:
90         eff = sum(connections_payoffs(G, c, delta, graph_type))
91     efficiencies = [eff]
92
93     for t in range(max_iters):
94         u = t % n
95
96         actions = get_possible_actions(G, u, graph_type)
97
98         best_payoff = -float('inf')
99         best_actions = []
100
101         for action in actions:
102             G_new = apply_action(G, u, action, graph_type)
103             if model == 'linear':
104                 payoff = linear_payoffs(G_new, c, graph_type)[u]
105             else:
106                 payoff = connections_payoffs(G_new, c, delta, graph_type)[u]
107
108             if payoff > best_payoff + 1e-9:
109                 best_payoff = payoff
110                 best_actions = [action]
111             elif abs(payoff - best_payoff) < 1e-9:
112                 best_actions.append(action)
113
114         none_in_best = False
115         for a in best_actions:
116             if a[0] == 'none':
117                 none_in_best = True
118                 break
119
120         if none_in_best:
121             chosen_action = ('none', None)
122         else:
123             chosen_action = best_actions[np.random.choice(len(best_actions))]
124
125         G = apply_action(G, u, chosen_action, graph_type)
126         history.append(tuple(sorted(G.edges())))
127
128         if model == 'linear':
129             eff = sum(linear_payoffs(G, c, graph_type))
130         else:
131             eff = sum(connections_payoffs(G, c, delta, graph_type))
132         efficiencies.append(eff)
133
134         if t >= n:
135             edges_same = True
136             for i in range(1, n+1):
137                 if history[-1] != history[-(i+1)]:
138                     edges_same = False
139                     break
140             if edges_same:
141                 break
142
143     return G, history, efficiencies
144
145 def calculate_efficiency(G, c, model, delta, graph_type):
146     if model == 'linear':
147         payoffs = linear_payoffs(G, c, graph_type)
148     else:
149         payoffs = connections_payoffs(G, c, delta, graph_type)

```



```

150     return sum(payoffs)
151
152 def generate_inits(n, graph_type):
153     inits = {}
154     if graph_type == 'undirected':
155         inits['Empty'] = nx.empty_graph(n)
156         inits['Complete'] = nx.complete_graph(n)
157         inits['Star'] = nx.star_graph(n-1)
158         inits['Cycle'] = nx.cycle_graph(n)
159     else:
160         inits['Empty'] = nx.empty_graph(n, create_using=nx.DiGraph)
161         inits['Complete'] = nx.complete_graph(n, create_using=nx.DiGraph)
162         star_out = nx.DiGraph()
163         star_out.add_nodes_from(range(n))
164         for i in range(1, n):
165             star_out.add_edge(0, i)
166         inits['Star_Out'] = star_out
167
168         star_in = nx.DiGraph()
169         star_in.add_nodes_from(range(n))
170         for i in range(1, n):
171             star_in.add_edge(i, 0)
172         inits['Star_In'] = star_in
173
174         cycle = nx.DiGraph()
175         cycle.add_nodes_from(range(n))
176         for i in range(n):
177             cycle.add_edge(i, (i+1)%n)
178         inits['Cycle'] = cycle
179     return inits
180
181 def plot_graph(G, title, filename):
182     plt.figure(figsize=(6,6))
183     pos = nx.spring_layout(G, seed=42)
184     nx.draw(G, pos, with_labels=True, node_color='lightblue', node_size=500, font_size=10,
185            arrows=isinstance(G, nx.DiGraph), edge_color='gray')
186     plt.title(title)
187     plt.savefig(f'/Users/mhanisch/projects/psets/q2/144/hw6/latex/figs/{filename}')
188     plt.close()
189
190 def plot_convergence(efficiencies, title, filename):
191     plt.figure(figsize=(6,4))
192     plt.plot(efficiencies, marker='o', markersize=3)
193     plt.xlabel('Iteration')
194     plt.ylabel('Total Efficiency')
195     plt.title(title)
196     plt.grid(True)
197     plt.savefig(f'/Users/mhanisch/projects/psets/q2/144/hw6/latex/figs/{filename}', bbox_inches='tight')
198     plt.close()
199
200 if __name__ == '__main__':
201     n = 9
202     out_lines = []
203
204     # 1. Undirected Linear
205     out_lines.append("=== Part 1: Undirected Linear ===")
206     c_vals = [0.5, 4.0, 10.0]
207     c_labels = ['c_lt_1', '1_lt_c_lt_n-1', 'c_gt_n-1']
208     for c, label in zip(c_vals, c_labels):
209         out_lines.append(f"\n\nRegime: {label} (c = {c})")
210         inits = generate_inits(n, 'undirected')
211         for init_name, init_G in inits.items():
212             G_eq, history, efficiencies = run_simulation(init_G, c, model='linear', graph_type='undirected')
213
214             eff = calculate_efficiency(G_eq, c, 'linear', delta=0, graph_type='undirected')
215             out_lines.append(f"  Init: {init_name}<10> | Steps: {len(history)-1:<4>} | Efficiency: {eff
216             :<6.2f} | Num Edges: {G_eq.number_of_edges()}")
217             plot_graph(G_eq, f"Undirected Linear, {label}, Init: {init_name}", f"undirected_linear_{label}
218             _{init_name}.png")
219             plot_convergence(efficiencies, f"Efficiency: Undirected Linear, {label}, Init: {init_name}", f
220             "conv_undirected_linear_{label}_{init_name}.png")
221
222     # 2. Directed Linear
223     out_lines.append("\n\n=== Part 2: Directed Linear ===")
224     for c, label in zip(c_vals, c_labels):
225         out_lines.append(f"\n\nRegime: {label} (c = {c})")

```

```

222     inits = generate_inits(n, 'directed')
223     for init_name, init_G in inits.items():
224         G_eq, history, efficiencies = run_simulation(init_G, c, model='linear', graph_type='directed')
225         eff = calculate_efficiency(G_eq, c, 'linear', delta=0, graph_type='directed')
226         out_lines.append(f" Init: {init_name:<10} | Steps: {len(history)-1:<4} | Efficiency: {eff
: <6.2f} | Num Edges: {G_eq.number_of_edges()}"")
227         plot_graph(G_eq, f"Directed Linear, {label}, Init: {init_name}", f"directed_linear_{label}_{
init_name}.png")
228         plot_convergence(efficiencies, f"Efficiency: Directed Linear, {label}, Init: {init_name}", f"
conv_directed_linear_{label}_{init_name}.png")
229
230 # 3. Undirected Connections
231 out_lines.append("\n=== Part 3: Undirected Connections ===")
232 delta = 0.5
233 # thresholds: delta - delta**2 = 0.25
234 # delta + (n/2-1)*delta**2 = 0.5 + (3.5)*0.25 = 1.375
235 c_vals_conn = [0.1, 0.8, 2.0]
236 c_labels_conn = ['c_small', 'c_medium', 'c_large']
237 for c, label in zip(c_vals_conn, c_labels_conn):
238     out_lines.append(f"\nRegime: {label} (c = {c})")
239     inits = generate_inits(n, 'undirected')
240     for init_name, init_G in inits.items():
241         G_eq, history, efficiencies = run_simulation(init_G, c, model='connections', delta=delta,
graph_type='undirected')
242         eff = calculate_efficiency(G_eq, c, 'connections', delta=delta, graph_type='undirected')
243         out_lines.append(f" Init: {init_name:<10} | Steps: {len(history)-1:<4} | Efficiency: {eff
: <6.2f} | Num Edges: {G_eq.number_of_edges()}"")
244         plot_graph(G_eq, f"Undirected Connections, {label}, Init: {init_name}", f"undirected_conn_{
label}_{init_name}.png")
245         plot_convergence(efficiencies, f"Efficiency: Undirected Connections, {label}, Init: {init_name
}", f"conv_undirected_conn_{label}_{init_name}.png")
246
247 with open('/Users/mhanisch/projects/psets/q2/144/hw6/code/simulate_networks.py.out', 'w') as f:
248     f.write("\n".join(out_lines))

```

solve_maze.py

```

1 import sys
2 import random
3 import matplotlib.pyplot as plt
4 import numpy as np
5 from maze import Maze
6
7 def get_next_state_intended(state, direction):
8     # Action map from maze: 0=N, 1=S, 2=E, 3=W
9     # but wait, the maze has moves explicitly defined:
10     moves = [(0,1), (0, -1), (1,0), (-1, 0)]
11     x_change = moves[direction][0]
12     y_change = moves[direction][1]
13
14     x, y = state
15     return ((x+x_change) % 10, (y+y_change) % 10)
16
17 def train_agent():
18     env = Maze()
19
20     # Parameters
21     alpha = 0.5
22     epsilon_start = 1.0
23     epsilon_min = 0.05
24     epsilon_decay = 0.995
25     epochs = 998
26
27     # Rewards / Valuations
28     R_unexplored = 0.0 # Standard plain room
29     R_snake = -100.0
30     R_exit = 100.0
31
32     # Initialize V table
33     V = {}
34     for x in range(10):
35         for y in range(10):
36             V[(x, y)] = R_unexplored
37
38     # Hardcode terminal states (so they never decay from their terminal value)
39     for snake in env.snakes:

```

```

40     V[sneak] = R_sneak
41     for exit in env.exits:
42         V[exit] = R_exit
43
44     epsilon = epsilon_start
45
46     training_steps = []
47
48     for epoch in range(epochs):
49         state = env.reset()
50         t = 0
51         while True:
52             # Epsilon greedy policy based on strictly adjacent rooms
53             if random.random() < epsilon:
54                 action = random.randint(0, 3)
55             else:
56                 best_val = -float('inf')
57                 best_actions = []
58                 for a in range(4):
59                     next_s = get_next_state_intended(state, a)
60                     val = V[next_s]
61                     if val > best_val:
62                         best_val = val
63                         best_actions = [a]
64                     elif val == best_val:
65                         best_actions.append(a)
66                 action = random.choice(best_actions)
67
68             next_state, reward_type = env.step(action)
69
70             # The environment step determines our actual next state.
71             # Update rule from the problem:  $V(i) = (1-\alpha)V(i) + \alpha * V(i')$ 
72             if state not in env.snakes and state not in env.exits:
73                 V[state] = (1 - alpha) * V[state] + alpha * V[next_state]
74
75             state = next_state
76             t += 1
77
78             if reward_type == 1 or reward_type == -1:
79                 break
80
81             epsilon = max(epsilon_min, epsilon * epsilon_decay)
82             training_steps.append(t)
83
84     return V, training_steps, env
85
86 def test_agent(V, env, episodes=1000):
87     success_count = 0
88     test_steps = []
89     for _ in range(episodes):
90         state = env.reset()
91         t = 0
92         success = False
93         while True:
94             # Greedy
95             best_val = -float('inf')
96             best_actions = []
97             for a in range(4):
98                 next_s = get_next_state_intended(state, a)
99                 val = V[next_s]
100                 if val > best_val:
101                     best_val = val
102                     best_actions = [a]
103                 elif val == best_val:
104                     best_actions.append(a)
105             action = random.choice(best_actions)
106
107             next_state, reward_type = env.step(action)
108             state = next_state
109             t += 1
110
111             if reward_type == 1:
112                 success_count += 1
113                 success = True
114                 break
115             elif reward_type == -1:

```

```

116         break
117
118         # Prevent infinite loops in testing
119         if t > 500:
120             break
121         test_steps.append(t)
122
123     return success_count, test_steps
124
125 def plot_valuations(V, env):
126     grid = np.zeros((10, 10))
127     for x in range(10):
128         for y in range(10):
129             grid[y, x] = V[(x, y)] # plotting y on rows, x on cols matches typical Cartesian grid if
                                     # inverted, but let's just use standard matrix
130
131     plt.figure(figsize=(10, 8))
132     # By passing origin='upper', row 0 is at Top.
133     # Col x, Row y matches array index grid[y, x]
134     im = plt.imshow(grid, cmap="RdYlGn")
135     plt.colorbar(im)
136     plt.title("Learned Room Valuations V(s)")
137     plt.xlabel("X Coordinate")
138     plt.ylabel("Y Coordinate")
139
140     # Annotate numbers
141     for x in range(10):
142         for y in range(10):
143             val = grid[y, x]
144             # Print integer part so "99.9" becomes "99" instead of rounding to "100.0"
145             plt.text(x, y, f"{int(val)}", ha='center', va='center', color='black' if -50 < val < 50 else '
white', fontsize=10)
146
147     # Annotate snakes and escapes
148     for (x, y) in env.snakes:
149         plt.text(x, y, 'S', ha='center', va='center', color='black', fontsize=16, fontweight='bold')
150     for (x, y) in env escapes:
151         plt.text(x, y, 'E', ha='center', va='center', color='blue', fontsize=16, fontweight='bold')
152
153     plt.text(0, 0, 'START', ha='center', va='center', color='purple', fontsize=12, fontweight='bold')
154
155     plt.tight_layout()
156     plt.savefig('../latex/figs/maze_learned_values.png')
157     plt.close()
158
159 if __name__ == "__main__":
160     V, t_steps, env = train_agent()
161     print("Training finished. Running test episodes...")
162     successes, test_steps = test_agent(V, env, 1000)
163     print(f"Successes: {successes}/1000")
164     if successes == 1000:
165         print("Agent successfully escapes 1000/1000 times!")
166
167     plot_valuations(V, env)
168     print("Saved valuation grid to latex/figs/maze_learned_values.png")

```